

Suggested Demo Flow

1. Half Adder

Files:

- `verilog/01_adders/half_adder.v`
- `verilog/01_adders/tb_half_adder.v`

Signals to discuss:

- `a`
- `b`
- `sum`
- `carry`

Main point: `sum` is XOR written as `a'b + ab'`, and `carry` is AND.

2. Full Adder

Files:

- `verilog/01_adders/full_adder.v`
- `verilog/01_adders/tb_full_adder.v`

Signals to discuss:

- `a`
- `b`
- `cin`
- `sum`
- `cout`

Main point: a full adder includes carry input, so it can be chained. The sum is written in SOP form, not using the Verilog XOR operator.

3. Ripple Carry Adder

Files:

- `verilog/02_ripple_carry_adder/full_adder.v`
- `verilog/02_ripple_carry_adder/ripple_carry_adder_4bit.v`
- `verilog/02_ripple_carry_adder/tb_ripple_carry_adder_4bit.v`

Signals to discuss:

- a
- b
- cin
- sum
- cout

Main point: four full adders are connected in series.

4. Multiplier

Files:

- verilog/03_multiplier/multiplier_4bit.v
- verilog/03_multiplier/tb_multiplier_4bit.v

Signals to discuss:

- a
- b
- product

Main point: multiplying two 4-bit numbers needs an 8-bit output.

5. ALU

Files:

- verilog/04_alu/alu_4bit.v
- verilog/04_alu/tb_alu_4bit.v

Signals to discuss:

- a
- b
- opcode
- result
- carry
- zero

Main point: the opcode is decoded using gates, and the result is selected using AND-OR multiplexer logic.